

计算理论与算法分析设计

总复习



CH1 算法概述

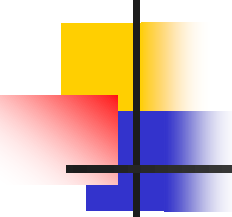




算法的五个特征

- 1) 有穷性 2) 确定性
3) 输入 4) 输出 5) 可行性

- **算法的定义:** Informally, an *algorithm* is any well-defined computational procedure that takes some value, or set of values, as *input* and produces some value, or set of values, as *output*. An algorithm is thus a sequence of computational steps that transform the input into the output.



O 、 o 、 Ω 、 ω 、 Θ 第一种理解方法

设 f 和 g 是定义域为自然数 N 上的函数

- $f(n) = O(g(n))$.

若存在正数 c 和 n_0 使得对一切 $n \geq n_0$ 有 $0 \leq f(n) \leq cg(n)$

- $f(n) = \Omega(g(n))$.

若存在正数 c 和 n_0 使得对一切 $n \geq n_0$ 有 $0 \leq cg(n) \leq f(n)$

- $f(n) = \Theta(g(n))$

$f(n) = O(g(n))$ 且 $f(n) = \Omega(g(n))$



■ 标准复杂性函数的比较

$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!) < O(n^n)$

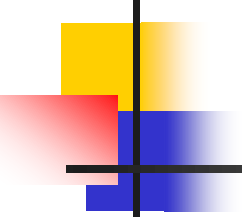
多项式时间阶

指数时间阶

一个算法的时间复杂性如果是 $O(n^k)$ (k 为有理数)，则称此算法需要多项式时间。

有效算法

以多项式时间为限界的算法称为有效算法



■ 标准复杂性函数的比较

$$\underbrace{O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3)}_{\text{多项式时间阶}} < \underbrace{O(2^n) < O(n!) < O(n^n)}_{\text{指数时间阶}}$$

多项式时间阶

指数时间阶

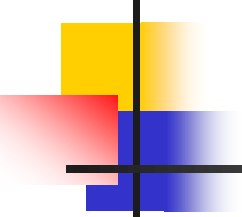
注意：1) 不能划等号 2) 以下若无特殊声明，

$\log$ 是以2为底的对数

3) 上式只有在 n 较大的时候成立

$O(1)$ 的含义？

计算时间由一个常数（零次多项式）来限界



例

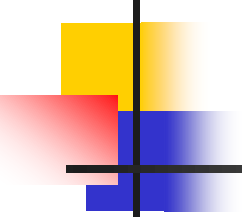
- 例：算法 A_1, A_2 的时间复杂性分别是 $n, 2^n$, 设 $100\mu s$ 是一个单位时间，求 A_1, A_2 在 $1s$ 内能处理的问题规模。
- 已知 $\lg 2 = 0.301$

$$T(n) = n$$

$$T(n) * 10^{-4} = 1$$

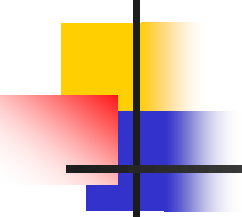
$$\text{即 } n * 10^{-4} = 1$$

$$\text{所以 } n = 10^4$$



例

- 假设某算法在输入规模为 n 的时间复杂性为 $T(n)=3*2^n$ 。在某台计算机上实现并完成该算法的时间为 t 秒。现在另一计算机，其运行速度为第一台的64倍，那么在这台新机器上用同一算法在 t 秒内能解输入规模为多大的问题？

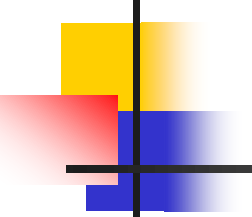


例

解：设新机器用同一算法内能解输入规模为 n_1 的问题，那么有 $3 * 2^n = 3 * 2^{n_1} / 64$ ，
解得 $n_1 = n + 6$.

CH2 分治法



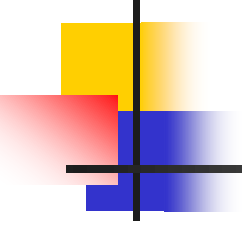


•例1 用分治法求n个元素集合S中的最大、最小元素。

•假设 $n=2^m$ 。要求每次平分成2个子集。

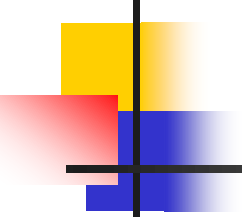
```
void maxmin(int A[],int &e_max,int &e_min,int low,int high)
```

```
2. {
3.     int mid,x1,y1,x2,y2;
4.     if ((high-low <= 1)) {
5.         if (A[high]>A[low]) {
6.             e_max = A[high];
7.             e_min = A[low];
8.         }
9.     } else {
10.         e_max = A[low];
11.         e_min = A[high];
12.     }
13. }
```



```
14.     else {  
15.         mid = (low + high) / 2;  
16.         maxmin(A, &x1, &y1, low, mid);  
17.         maxmin(A, &x2, &y2, mid+1, high);  
18.         e_max = max(x1, x2);  
19.         e_min = min(y1, y2);  
20.     }  
21. }
```

$$T(n) = \begin{cases} 1 & n=2 \\ 2T(n/2) + 2 & n>2 \end{cases}$$



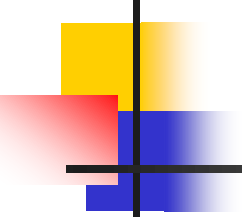
用分治法求n个元素集合S中的最大、最小元素。写出算法，并分析时间复杂性（比较次数）。

假设 $n=3^m$ 。要求每次平分成3个子集。

$$T(n) = \begin{cases} 3 & n=3 \\ 3T(n/3)+4 & n>3 \end{cases}$$

$$T(n) = 5n/3 - 2$$

平分成2个子集 $T(n) = 3n/2 - 2$



归并排序 (Merge Sort)

```
void MergeSort(int A[], int B[], int l, int h)
```

```
{
```

```
    if(l == h)
```

```
        return;
```

```
    int m = (l+h)/2;
```

```
    MergeSort(A, B, l, m);
```

```
    MergeSort(A, B, m+1, h);
```

```
    Merge(A, B, l, m, h);
```

```
}
```

$$T(n) = \begin{cases} 1 & n=2 \\ 2T(n/2) + cn & n>2 \end{cases}$$

主定理：其中 $n = c^k$, k 为某个非负常数

$$T(n) = \begin{cases} d & n=2 \\ aT(n/c) + bn^x & n>2 \end{cases}$$

$$T(n) = \begin{cases} \Theta(n^x) & a < c^x \\ \Theta(n^x \log n) & a = c^x \\ \Theta(n^{\log_c a}) & a > c^x \end{cases}$$

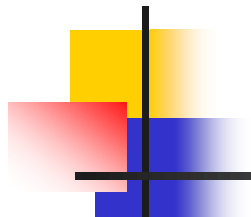
$$T(n) = \begin{cases} 1 & n=2 \\ 2T(n/2) + cn & n>2 \end{cases}$$

■ 归并排序
 $\Theta(n \log n)$

$$T(n) = \begin{cases} 1 & n=2 \\ 2T(n/2) + 2 & n>2 \end{cases}$$

快排序---划分过程

0	1	2	3	4	5	6	7	
low							high	
49	38	65	97	76	13	27	49	
						high		pivot = 49
27	38	65	97	76	13		49	
		low						
27	38		97	76	13	65	49	
					high			
27	38	13	97	76		65	49	
			low					
27	38	13	49	76	97	65	49	
			high					
			=low					



大整数乘法

由 $X=A2^{n/2}+B$, $Y=C2^{n/2}+D$ 则

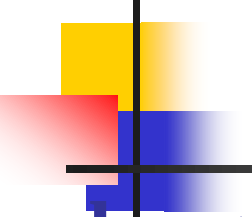
$$\begin{aligned} XY &= (A2^{n/2}+B)(C2^{n/2}+D) = AC2^n + (AD+BC)2^{n/2} + BD \\ &= AC2^n + ((A-B)(D-C) + AC + BD)2^{n/2} + BD \end{aligned}$$

计算成本：3次 $n/2$ 位乘法，6次不超过 n 位加减法，2次移位，所有加法和移位共计 $O(n)$ 次运算。由此可得，

$$\begin{cases} T(1) = 1 \\ T(n) = 3T(n/2) + cn \end{cases}$$

$$T(n) = O(n^{\log 3}) = O(n^{1.59})$$

这种思想同样可以用于十进制数的乘法中。



求第k小的元素

long Select(k,S)

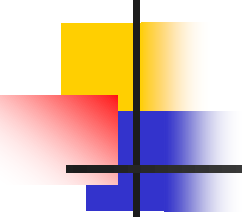
{ if ($|S| \leq 38$)

{ 将S中的元素排成非递减序;

return (S中的第k小元素); }

else

{ 将S中的元素划分成长度等于5的
 $\lfloor |S|/5 \rfloor$ 个子序列;



由各子序列的中值元素组成非递减序列 M ;

$m \leftarrow \text{Select}(\lceil |M|/2 \rceil, M);$

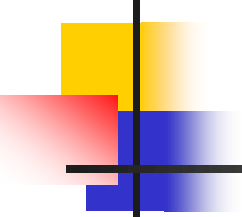
按 m 将 S 中的元素划分成小于 m 、等于

m 和大于 m 的三个子序列 S_1 , S_2 和 S_3 ;

if ($|S_1| > k$) return($\text{Select}(k, S_1)$);

else if ($|S_1| + |S_2| \geq k$) return(m);

else return($\text{Select}(k - |S_1| - |S_2|, S_3)$); }



线性时间选择问题:

- 用线性时间从 n 个元素中选出第 k 个小的元素
- 定理: 算法 **Select** 在 $O(n)$ 时间内找出 n 个元素序列中的第 k 小的元素。

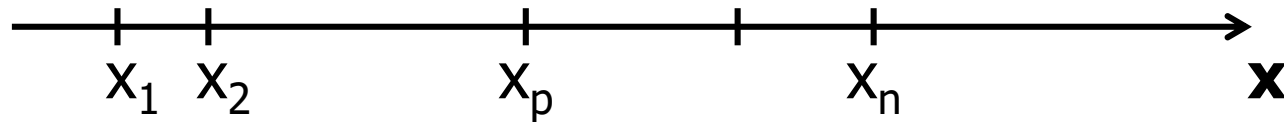
$$T(n) \leq \begin{cases} c & n \leq 38 \\ T(\lfloor n/5 \rfloor) + T(\lfloor 3n/4 \rfloor) + cn & n > 38 \end{cases}$$

$$T(n) = 20cn$$

线性时间选择: 中位数应用

■ 中位数原理

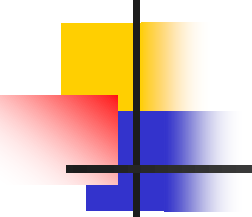
X轴上有n个点，由左至右依次排列为



找一个点 x_p (不一定是n个点之一), 使 x_p 到各点距离和最小, 解为:

$$x_p = \begin{cases} x_{(n+1)/2} & \text{当 } n \text{ 为奇数时} \\ \text{中间两个点的闭区间上} & \text{当 } n \text{ 为偶数时} \end{cases}$$

即解为中位数或包含中位数的闭区间。



最大子段和: 分治算法

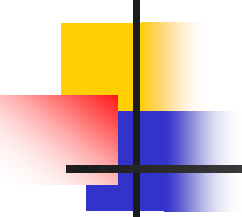
● 基本思想

将 $A[1..n]$ 分为 $a[1..n/2]$ 和 $a[n/2+1..n]$, 分别对两区段求最大子段和, 这时有三种情形:

Case 1: $a[1..n]$ 的最大子段和的子段落在 $a[1..n/2]$;

Case 2: $a[1..n]$ 的最大子段和的子段落在 $a[n/2..n]$;

Case 3: $a[1..n]$ 的最大子段和的子段跨在 $a[1..n/2]$ 和 $a[n/2..n]$ 之间;

- 
- 对 Case 1 和 Case 2 可递归求解；

对 Case 3，可知 $a[n/2]$ 和 $a[n/2+1]$ 一定在最大和的子段中，因此

在 $a[1..n/2]$ 中计算：

$$S_1 = \max_{1 \leq i \leq n/2} \sum_{k=i}^{n/2} a_k$$

在 $a[n/2..n]$ 中计算：

$$S_2 = \max_{n/2+1 \leq i \leq n} \sum_{k=n/2+1}^i a_k$$

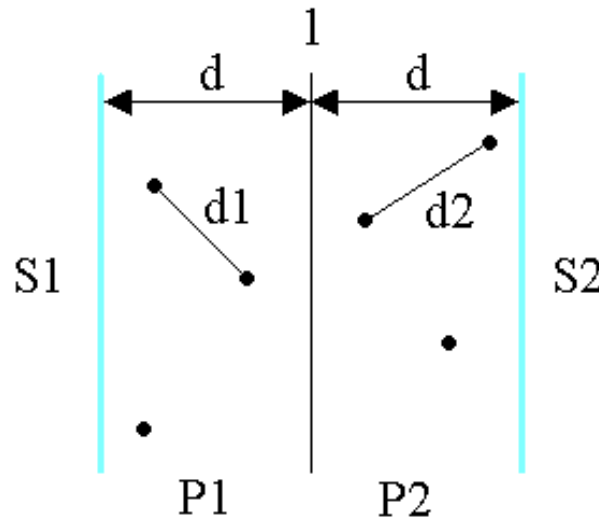
易知： $S_1 + S_2$ 是 Case 3 的最大值

最接近点对问题

选取一垂直线 $l:x=m$ 来作为分割直线。其中 m 为 S 中各点 x 坐标的中位数。由此将 S 分割为 $S1$ 和 $S2$ 。

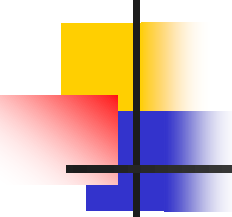
递归地在 $S1$ 和 $S2$ 上找出其最小距离 $d1$ 和 $d2$ ，并设 $d=\min\{d1,d2\}$ ， S 中的最接近点对或者是 d ，或者是某个 $\{p,q\}$ ，其中 $p \in P1$ 且 $q \in P2$ 。

在线性时间内找到 p,q



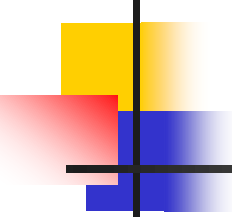
CH3 动规





方法概述：基本思想

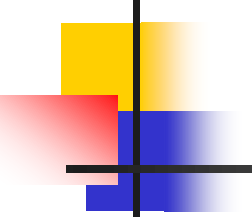
- 动态规划的思想实质是分治思想和解决冗余。
- 与分治法类似的是，将原问题分解成若干个子问题，先求解子问题，然后从这些子问题的解得到原问题的解。
- 与分治法不同的是，经分解的子问题往往不是互相独立的。若用分治法来解，有些共同部分（子问题或子子问题）被重复计算了很多次。



方法概述: 适用条件

动态规划法的有效性依赖于问题本身所具有的
两个重要性质

- **最优子结构**: 当问题的最优解包含了其子问题的最优解时, 称该问题具有最优子结构性质。
- **重叠子问题**: 在用递归算法自顶向下解问题时, 每次产生的子问题并不总是新问题, 有些子问题被反复计算多次。动态规划算法正是利用了这种子问题的重叠性质, 对每一个子问题只解一次, 而后将其解保存在一个表格中, 在以后尽可能多地利用这些子问题的解。



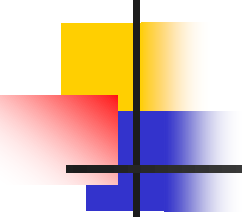
自底向上的动规与备忘录区别

- 自底向上的动规

每个子问题至少求解一次.

- 备忘录

某些子问题根本没有必要求解.



矩阵链乘问题

矩阵链乘问题满足最优性原理

记 $A[i:j]$ 为 $A_i A_{i+1} \dots A_j$ 链乘的一个最优括号方案，设 $A[i:j]$ 的最优次序中含有二个子链 $A[i:k]$ 和 $A[k+1:n]$ ，则 $A[i:k]$ 和 $A[k+1:n]$ 也是最优的。（反证可得）

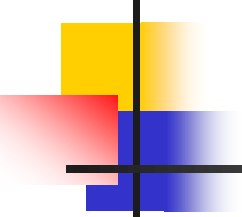
矩阵链乘问题

● 递归求解最优解的值

记 $m[i][j]$ 为计算 $A[i:j]$ 的最少乘法数，则原问题的最优值为 $m[1][n]$

$$m[i][j] = \begin{cases} 0 & i = j \\ \min_{i \leq k < j} \{ m[i][k] + m[k+1][j] + p_{i-1} p_k p_j \} & i < j \end{cases}$$

$\underbrace{\hspace{10em}}_{(A_i A_{i+1} \cdots A_k)_{p_{i-1} \times p_k} \times (A_{k+1} A_{k+2} \cdots A_j)_{p_k \times p_j}}$



0-1背包问题

- n 个物品，背包载重量为 C
- 递归求解最优解的值

记 $p_i(j)$ 为前 i 个物品装入载重量为 j 的背包的最大价值，则原问题的最优值为 $p_n(C)$

$$p_i(j) = \begin{cases} p_{i-1}(j) & j < w_i \\ \max\{p_{i-1}(j), p_{i-1}(j - w_i) + p_i\} & j \geq w_i \end{cases}$$

0-1背包问题

	0	$j - w_i$	j	M
0	0	0	0	0
$i - 1$	0	$p_{i-1}(j - w_i)$	$p_{i-1}(j)$	
i	0		$p_i(j)$	
n	0			目标

最长公共子序列的结构

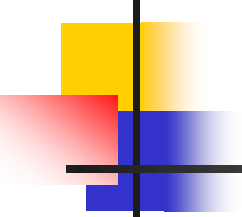
设序列 $X=\{x_1, x_2, \dots, x_m\}$ 和 $Y=\{y_1, y_2, \dots, y_n\}$ 的最长公共子序列为 $Z=\{z_1, z_2, \dots, z_k\}$ ，则

- (1) 若 $x_m = y_n$ ，则 $z_k = x_m = y_n$ ，
且 $z_1, z_2, \dots, z_{k-1}$ 是否为 $x_1, x_2, \dots, x_{m-1}$ 和 $y_1, y_2, \dots, y_{n-1}$ 的最长公共子序列。
- (2) 若 $x_m \neq y_n$ 且 $z_k \neq x_m$ ，
则 Z 是 $x_1, x_2, \dots, x_{m-1}$ 和 Y 的最长公共子序列
- (3) 若 $x_m \neq y_n$ 且 $z_k \neq y_n$ ，
则 Z 是 X 和 $y_1, y_2, \dots, y_{n-1}$ 的最长公共子序列

子问题的递归结构

由最长公共子序列问题的最优子结构性质建立子问题最优值的递归关系。用 $c[i][j]$ 记录序列 X 和 Y 的最长公共子序列的长度。其中， $X_i = \{x_1, x_2, \dots, x_i\}$;
 $Y_j = \{y_1, y_2, \dots, y_j\}$ 。当 $i=0$ 或 $j=0$ 时，空序列是 X_i 和 Y_j 的最长公共子序列。故此时 $C[i][j]=0$ 。其它情况下，由最优子结构性质可建立递归关系如下：

$$c[i][j] = \begin{cases} 0 & i = 0, j = 0 \\ c[i-1][j-1] + 1 & i, j > 0; x_i = y_j \\ \max\{c[i][j-1], c[i-1][j]\} & i, j > 0; x_i \neq y_j \end{cases}$$



最长递增子序列(LIS)

输入: 实数序列 $(x_1, x_2, \dots, x_n)$

输出: $x_{i_1} \leq x_{i_2} \leq \dots \leq x_{i_k}$, 其中 k 最大且 $i_1 \leq i_2 \leq \dots \leq i_k$.

• 样例: (2,8,9,4,6,1,3,7,5,10) : (2,4,6,7,10)

• $O(n^2)$ 算法

$dp[i]$ 表示以 a_i 结尾的最长递增子序列长度。

$dp[i] = \max(dp[j] + 1)$ 满足 $a[j] \leq a[i]$

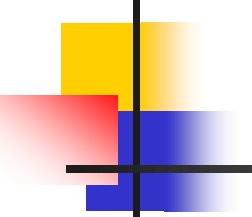
$1 \leq i \leq n-1$ $0 \leq j \leq i-1$ 初值所有 $dp[i] = 1$

• $O(n \log n)$ 算法

利用二分

CH4 贪心法





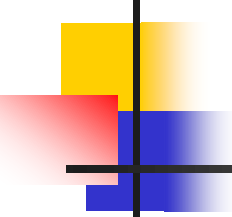
部分(或者小数)背包问题

已知一个容量大小为 M 重量的背包和 n 种物品，物品 i 的重量为 w_i ，假定物品 i 的一部分 x_i 放入背包会得到 $v_i x_i$ 这么大的收益，这里， $0 \leq x_i \leq 1$ ， $v_i > 0$ 。采用怎样的装包方法才会使装入背包的物品总效益最大？

例：考虑以下情况下的背包问题

$n=3$, $M=20$, $(v_1, v_2, v_3)=(25,24,15)$

$(w_1, w_2, w_3)=(18,15,10)$


$$n=3, M=20, (v_1, v_2, v_3) = (25, 24, 15)$$

$$(w_1, w_2, w_3) = (18, 15, 10)$$

按 v_i/w_i 的非增次序将物品依次放入背包

$$(x_1, x_2, x_3)$$

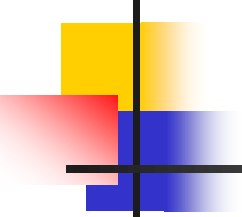
$$\sum w_i x_i$$

$$\sum v_i x_i$$

$$(0, 1, 1/2)$$

$$20$$

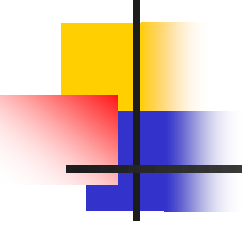
$$31.5$$



活动安排: 问题描述

有 n 个活动集 $E=\{1, 2, \dots, n\}$ 使用同一资源，而同一时间内同一资源只能由一个活动使用。每个活动的使用时间为 $[s_i, f_i)$ $i=1, \dots, n$ ， s_i 为开始时间， f_i 为结束时间，若 $[s_i, f_i)$ 与 $[s_j, f_j)$ 不相交称活动 i 和活动 j 是相容的。

问题：选出最大的相容活动子集合。



● 贪心策略 将各活动按结束时间排序 $f_1 \leq f_2 \leq \dots \leq f_n$, 先选出活动 1, 然后按活动编号从小到大的次序依次选择与当前活动相容的活动。

活动安排：计算示例

11个活动已按结束时间排序，用贪心算法求解：

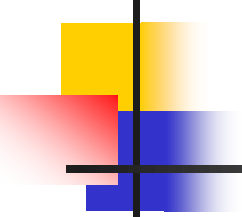
i	1	2	3	4	5	6	7	8	9	10	11
start_time _i	1	3	0	5	3	5	6	8	8	2	12
finish_time _i	4	5	6	7	8	9	10	11	12	13	14

time	a ₁	a ₂	a ₃	a ₄	a ₅	a ₆	a ₇	a ₈	a ₉	a ₁₀	a ₁₁
0											
1											
2											
3											
4											
5											
6											
7											
8											
9											
10											
11											
12											
13											
14											

相容活动：{a₃, a₉, a₁₁},
{a₁, a₄, a₈, a₁₁}, {a₂, a₄, a₉, a₁₁}



time	a ₁	a ₂	a ₃	a ₄	a ₅	a ₆	a ₇	a ₈	a ₉	a ₁₀	a ₁₁
0											
1											
2											
3											
4											
5											
6											
7											
8											
9											
10											
11											
12											
13											
14											

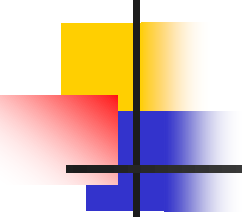


最优装载

轮船载重为 c , 集装箱重量为 w_i ($i=1,2,\dots,n$), 在装载体积不受限制的情况下, 尽可能多的集装箱装上轮船。

问题的形式化定义:

$$\begin{aligned} & \max \sum_{i=1}^n x_i \\ & \text{s.t.} \begin{cases} \sum_{i=1}^n w_i x_i \leq c & w_i > 0 \\ x_i \in \{0,1\} & i = 1,2,\dots,n \end{cases} \end{aligned}$$



假设 $n = 8$,

$[w_1, \dots, w_8] = [100, 200, 50, 90, 150, 50, 20, 80]$, $c = 400$ 。

从剩下的货箱中，选择重量最小的货箱。

排序之车间作业计划模型

一台机器、n个零件的排序

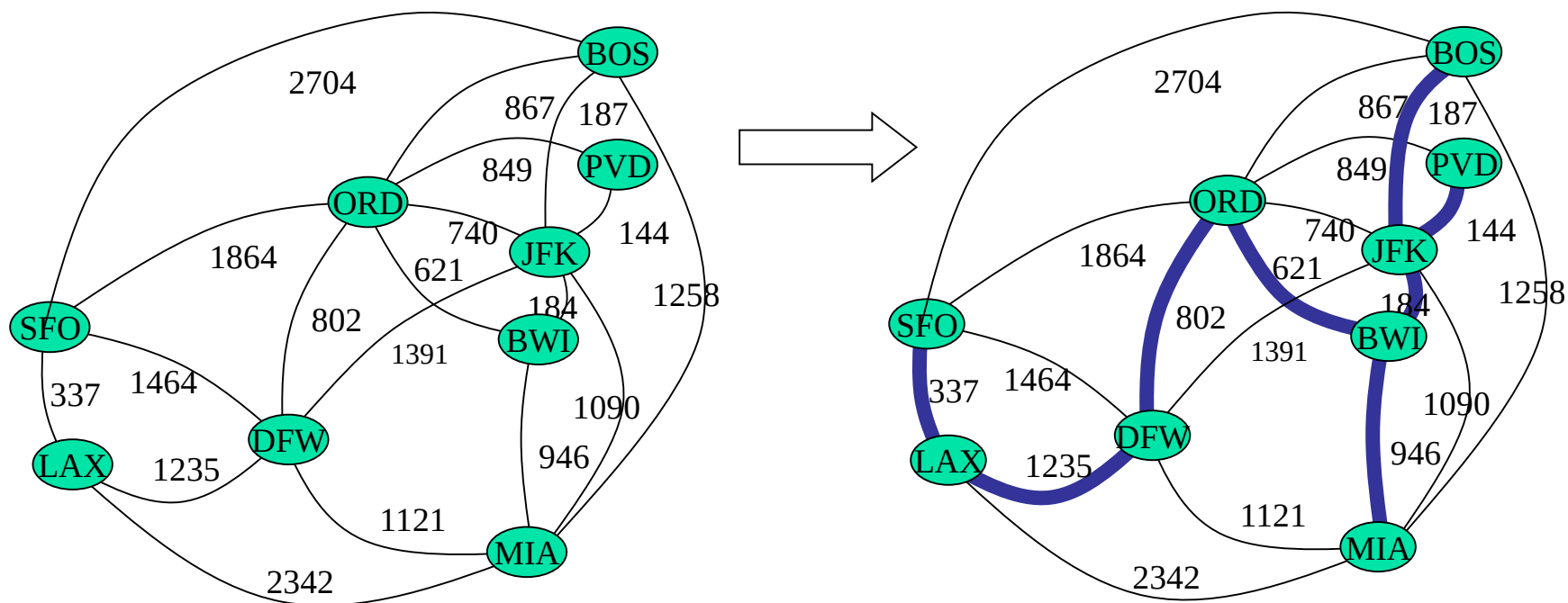
目的：使得各加工零件在车间里停留的平均时间最短。

零件	加工时间
1	1.8
2	2.0
3	0.5
4	0.9
5	1.3
6	1.5

最短：3, 4, 5, 6, 1, 2
 $(0.5+1.4+2.7+4.2+6+8)/6=3.8$

最小生成树(minimum spanning tree)

- 无向连通带权图 $G=(V,E,w)$.
- G 的**生成树**是 G 的包含所有顶点的一颗子树
- 若 G 的生成树 T' 在所有 G 的生成树中各边权总和最小, 则 T' 称为 G 的最小生成树(MST)



Prim算法 $O(|V|^2)$, $O(|E|\log|V|)$

• 输入: $G=(V,E,w)$ 无向连通; 输出: G 的MST

• 维护 $S \subseteq V$, 初始任取 V 中一点 r

当 $S \neq V$, 取 S 到 $V-S$ 的最小权边 (u,v) , 将 v 加入 S .

$key[u]$ 记 u 到 S 的最小距离, $G[u]$ 记 u 与 S 最小距离对应点

1. 初始 $G[u]=NIL$, $key[r]=0$, 其它 $key[u]=INF$, $Q=V$, S 空

2. 当 Q 非空

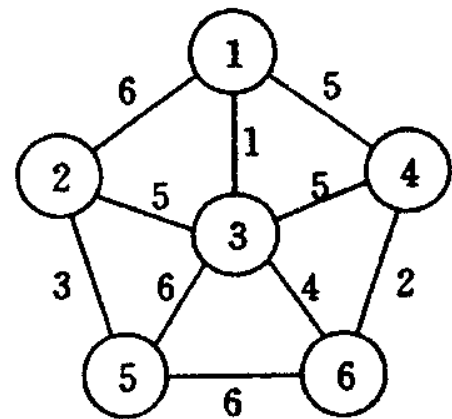
3. 取出 Q 中 u 使得 $key[u]$ 最小, 加入 S

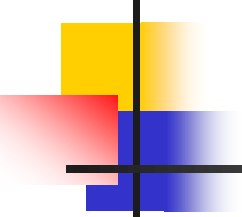
4. 对 u 的每个邻居 v ,

5. 若 $v \in Q$ 且 $w[u,v] < key[v]$

6. 则 $key[v] = w[u,v]$, $G[v] = u$

Q 一般数组 $O(|V|^2+|E|)$, Q 优先队列 $O(|V|\log|V|+|E|\log|E|)$





并查集算法(Make-set, Find-set)

$p[x]$: x 的父亲; $\text{rank}[x]$: x 的阶; $\text{Find}(x)$: 找 x 的根

Make(x)

1 $p[x]=x$

2 $\text{rank}[x]=0$

Union(x,y)

1 $\text{Link}(\text{Find}(x), \text{Find}(y))$

Link(x,y) //合并两树根

1 若 $\text{rank}[x] > \text{rank}[y]$

2 则 $p[y]=x$

3 否则 $p[x]=y$

4 若 $\text{rank}[x] = \text{rank}[y]$

5 则 $\text{rank}[y] = \text{rank}[y] + 1$

树根阶 r
 $\Rightarrow$
节点数
 $\geq 2^r$.

Find(x)

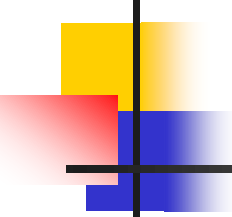
1 若 $x \neq p[x]$, 则

2 $p[x] = \text{Find}(p[x])$

3 返回 $p[x]$

路径压缩(pc)技术

n 个节点, m 次操作. 不计pc: $O(m \log n)$; 计pc: $O(m \alpha(n))$



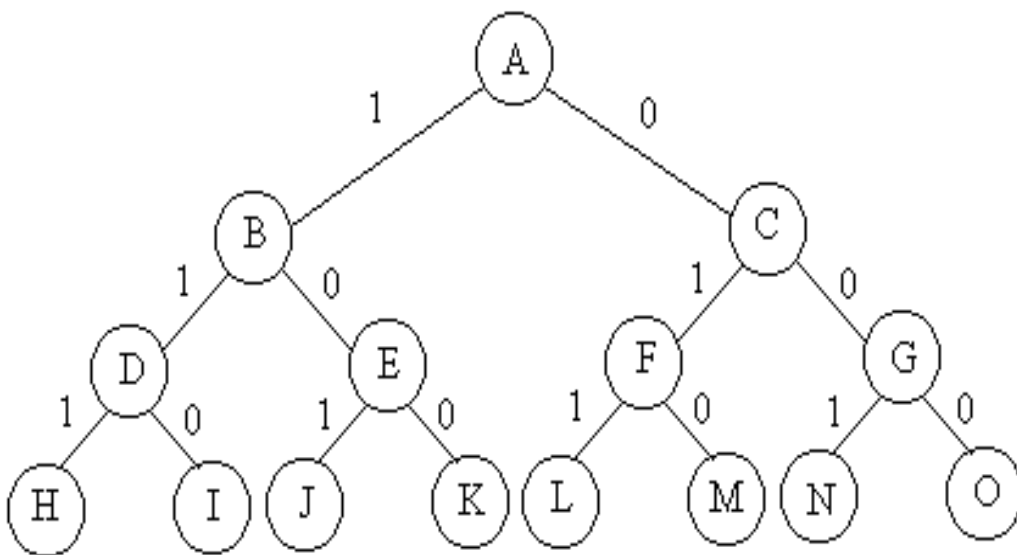
加入并查集结构的Kruskal算法

1. A为空, Q=E按边权升序排列, 每个点是一颗树
 2. 当Q非空
 3. 顺序取Q中边(u,v)
 4. 若u,v在不同树中, 则添(u,v)到A, 合并u,v所在树,
 5. 输出A
-

1. A为空, Q=E按边权升序排列, $\forall x$ Make(x)
2. 当Q非空
3. 顺序取Q中边(u,v)
4. 若Find(u) $\neq$ Find(v), 则添(u,v)到A, Union(u,v),
5. 输出A

CH5 回溯法

子集树与排列树

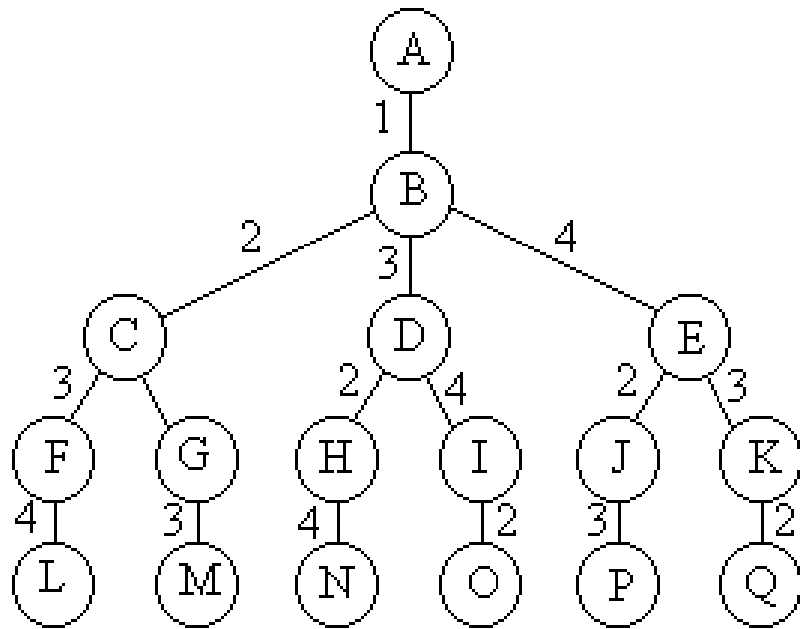


遍历子集树需 $O(2^n)$

计算时间

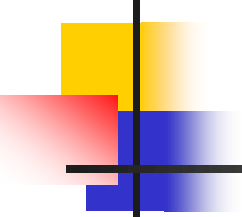
```
void backtrack (int t)
{
    if (t>n) output(x);
    else
        for (int i=0;i<=1;i++)
        {
            x[t]=i;
            if (legal(t))
                backtrack(t+1);
        }
}
```

子集树与排列树



遍历排列树需要
 $O(n!)$ 计算时间

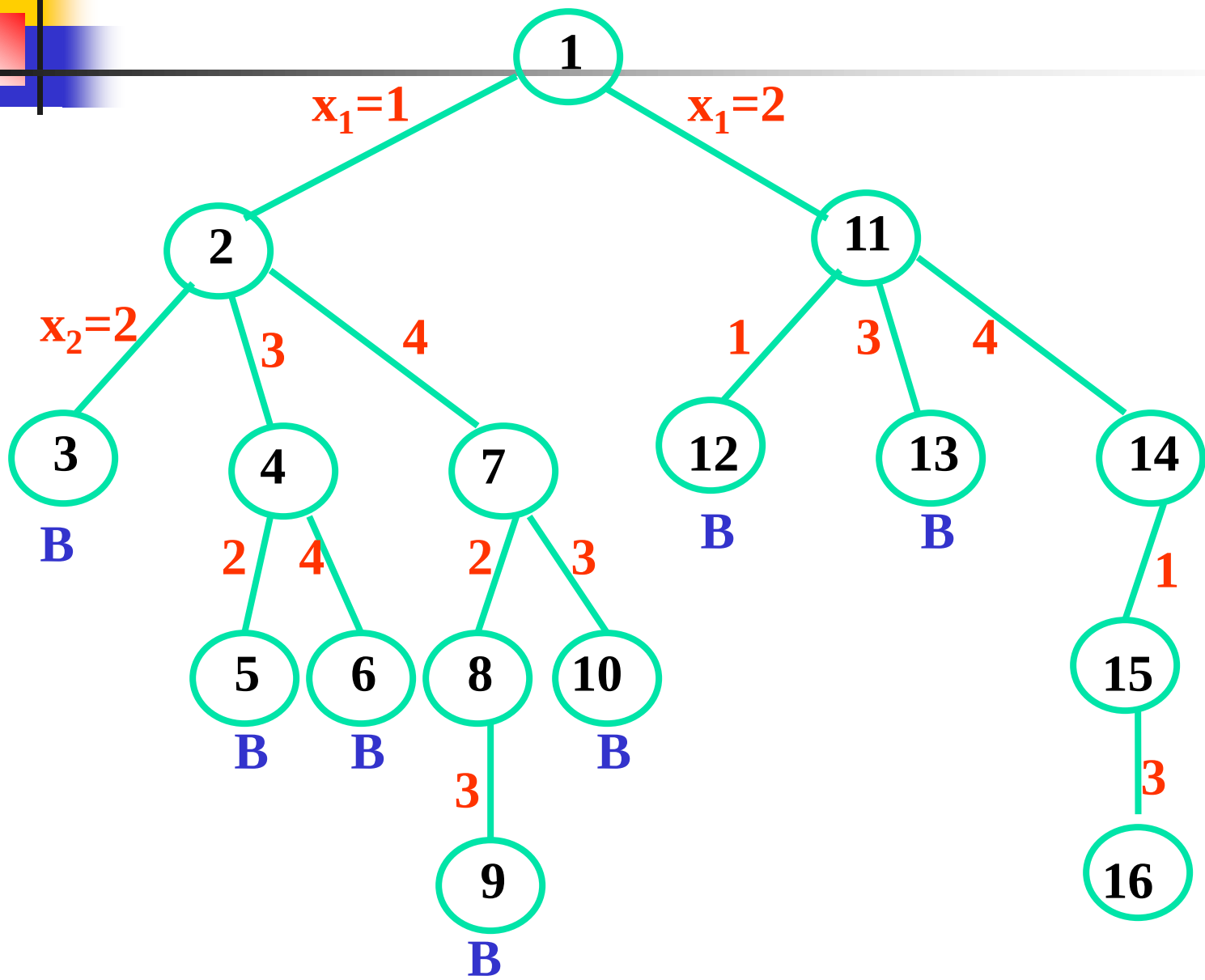
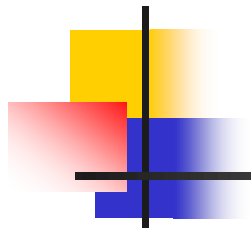
```
void backtrack (int t)
{
    if (t>n) output(x);
    else
        for (int i=t;i<=n;i++)
        {
            swap(x[t], x[i]);
            if (legal(t))
                backtrack(t+1);
            swap(x[t], x[i]);
        }
}
```

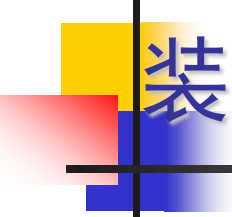


4后问题

设有一**4*4**的棋盘，把**4**个皇后放在棋盘上，
要求满足下列两个条件：

- 1) 任意两个皇后不在同一行上和同一列上；
 - 2) 任意两个皇后不在同一条对角线上；
- 问有多少种放法？





装载问题

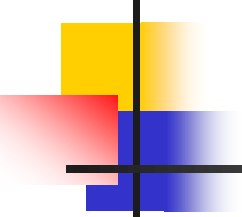
有一批共 n 个集装箱要装上2艘载重量分别为 c_1 和 c_2 的轮船，其中集装箱 i 的重量为 w_i ，且

$$\sum_{i=1}^n w_i \leq c_1 + c_2$$

装载问题要求确定是否有一个合理的装载方案可将这个集装箱装上这2艘轮船。如果有，找出一种装载方案。

最优装载方案：

- (1) 首先将第一艘轮船尽可能装满；**
- (2) 将剩余的集装箱装上第二艘轮船。**



问题分析

- 将第一艘轮船尽可能装满等价于选取全体集装箱的一个子集，使该子集中集装箱重量之和最接近。

$$\max \sum_{i=1}^n w_i x_i$$

$$\text{s.t.} \sum_{i=1}^n w_i x_i \leq c_1$$

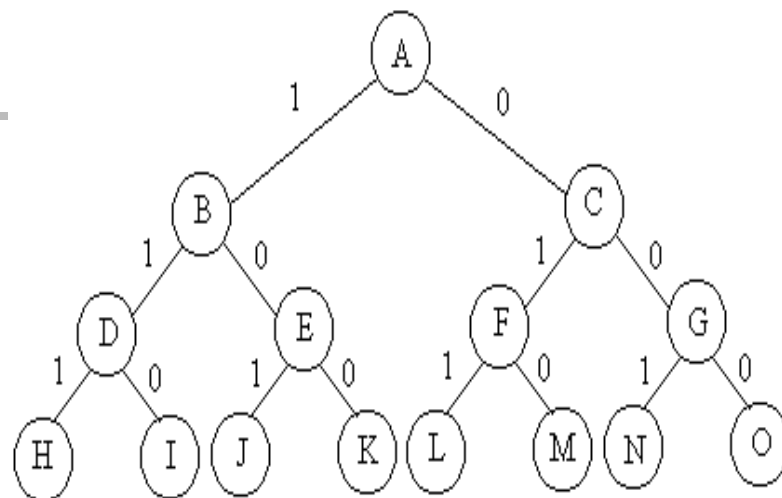
$$x_i \in \{0,1\}, 1 \leq i \leq n$$

- 由此可知，**装载问题等价于以下特殊的0-1背包问题。**

- 解空间：子集树
- 可行性约束函数(选择当前元素):

$$\text{s.t. } \sum_{i=1}^n w_i x_i \leq c_1$$
$$x_i \in \{0, 1\}, 1 \leq i \leq n$$

- 上界函数
 - 当前载重量 cw +剩余集装箱的重量 $r \leq$ 当前最优载重量 $bestw$



■ 上界函数:

- 用于剪去不含最优解的子树，从而提高算法在平均情况下的运行效率。
- 设Z是解空间树第i层上的当前扩展结点，**cw**是当前载重量，**R**是剩余集装箱的重量，**bestW**是当前最优载重量，则当
 $cw+r \leq bestW$ 时，剪去Z的右子树。

装载问题

• 解空间：子集树

• 可行性约束函数(选择当前元素):

• 上界函数(不选择当前元素):

$$\sum_{i=1}^n w_i x_i \leq c_1$$

当前载重量**cw**+剩余集装箱的重量**r**≤当前最优载重量**bestw**

```
private static void backtrack (int i)
```

```
{// 搜索第i层结点
```

```
    if (i > n) { // 到达叶结点
```

```
        //更新最优解bestx,bestw;return;
```

```
        if(cw>bestw){
```

```
            for(j=1; j<=n; j++) bestx[j]=x[j]; bestw = cw; }
```

```
        return; }
```

```
    r -= w[i];
```

```
    if (cw + w[i] <= c) {// 搜索左子树
```

```
        x[i] = 1;
```

```
        cw += w[i];
```

```
        backtrack(i + 1);
```

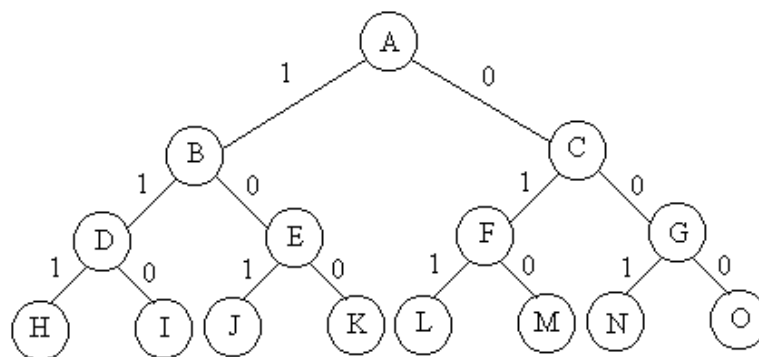
```
        cw -= w[i]; }
```

```
    if (cw + r > bestw) {
```

```
        x[i] = 0; // 搜索右子树
```

```
        backtrack(i + 1); }
```

```
    r += w[i];
```



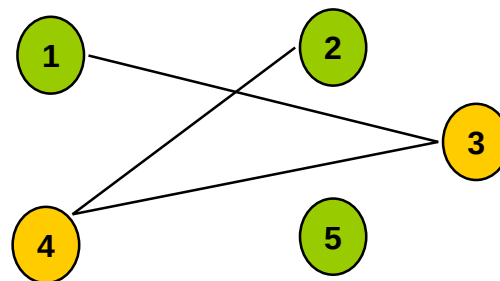
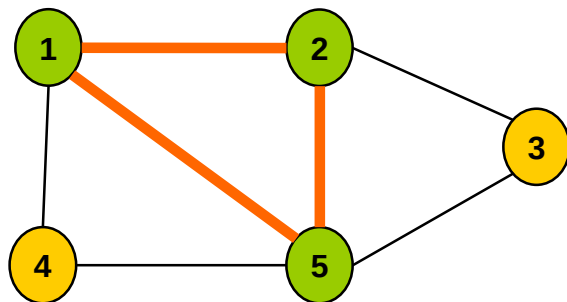
最大团问题

给定无向图 $G=(V, E)$ 。如果 $U \subseteq V$ ，且对任意 $u, v \in U$ 有 $(u, v) \in E$ ，则称 U 是 G 的完全子图。 G 的完全子图 U 是 G 的团。 G 的最大团是指 G 中所含顶点数最多的团。

如果 $U \subseteq V$ 且对任意 $u, v \in U$ 有 $(u, v) \notin E$ ，则称 U 是 G 的空子图。 G 的空子图 U 是 G 的独立集当且仅当 U 不包含在 G 的更大的空子图中。 G 的最大独立集是 G 中所含顶点数最多的独立集。

对于任一无向图 $G=(V, E)$ 其补图 $\bar{G}=(V_1, E_1)$ 定义为： $V_1=V$ ，且 $(u, v) \in E_1$ 当且仅当 $(u, v) \notin E$ 。

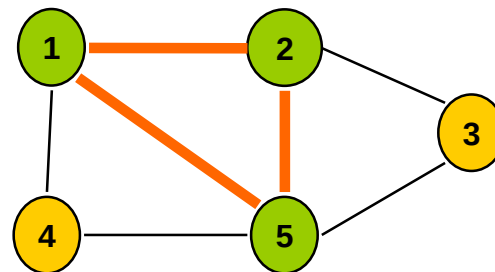
U 是 G 的最大团当且仅当 U 是 $\bar{G}$ 的最大独立集。



最大团问题

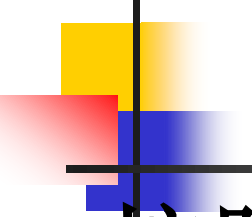
- 解空间：子集树
- 可行性约束函数：顶点 i 到已选入的顶点集中每一个顶点都有边相连。
- 上界函数：有足够多的可选择顶点使得算法有可能在右子树中找到更大的团。

```
private static void backtrack(int i)
{
    if (i > n) { // 到达叶结点
        for (int j = 1; j <= n; j++) bestx[j] = x[j];
        bestn = cn; return;
    }
    // 检查顶点 i 与当前团的连接
    boolean ok = true;
    for (int j = 1; j < i; j++)
        if (x[j] == 1 && !a[i][j]) { // i与j不相连
            ok = false; break;
        }
    if (ok) { // 进入左子树
        x[i] = 1; cn++;
        backtrack(i + 1);
        cn--;
    }
    if (cn + n - i > bestn) { // 进入右子树
        x[i] = 0;
        backtrack(i + 1);
    }
}
```



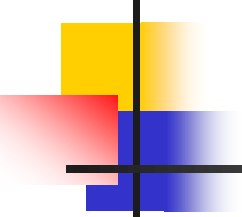
■ 问题

- 给定由 n 个不同正数组成的集合 $W = \{w_i\}$, 和正数 M , 求 W 中所有和等于 M 的子集的集合;
- 例如 $n = 6, M = 30$,
 $W = \{10, 13, 5, 18, 12, 15\}$



子集和问题

- 按照回溯法思想，从状态树的根结点出发，做深度优先搜索；
- 当在某一状态A下，依次尝试加入和不加入正数 w_i ，若 $\sum A + w_i > M$ ，则可停止对该结点的搜索；若 $\sum A + \sum(w_i \dots w_n) < M$ ，则也可停止对该结点的搜索；



回溯过程的效率

- 用回溯法去处理一实例所要生成的结点数，一般是采用在状态空间树中生成一条随机路径的方法估计。